Lecture 1: Hashing

1 Dictionaries and Hashing

A dictionary stores keys and their associated values. A compiler might store the type of each variable, while a website might store the account associated with each username.

We want to support three operations:

- INSERT(x, v): store or update the value associated with x.
- SEARCH(x): return its value, or report that x is absent.
- DELETE(x): remove x if it is present.

Why not use an array? Suppose the keys are integers from 0 to 10^{12}, but only one million keys are present. What is the disadvantage of using the key itself as an array index?

A *hash function* maps a large universe into a smaller set of buckets:

$$h : U \longrightarrow \{0, 1, \ldots, B-1\}.$$

We use $h(x)$ to decide where to store the key x.

Question 1. If $x \neq y$ but $h(x) = h(y)$, which conclusion must follow?

(a) The dictionary gives an incorrect answer

(b) The dictionary must have a way to distinguish x from y

(c) No dictionary can store both keys

For our dictionary bounds, assume that keys occupy $O(1)$ machine words, key comparison takes $O(1)$ time, and evaluating h takes $O(1)$ time. We will account separately for the cost of processing long strings.

Where does randomness enter?

Instead of using one fixed hash function for every input, we can choose a function randomly when building the table. The resulting dictionary still compares actual keys before returning an answer.

Monte Carlo algorithms A Monte Carlo algorithm has a prescribed running-time bound, but may return an incorrect answer. For each fixed input x, we want

$$\Pr[A(x) \neq f(x)] \leq \delta.$$

The probability is over the random choices of the algorithm.

Las Vegas algorithms A Las Vegas algorithm always returns the correct answer, but its running time is a random variable. We usually bound its expected running time. Randomized quicksort is one example.

Question 2. A hash-table implementation compares full keys, but randomly chooses its hash function. What can a collision change?

(a) Whether the answer is correct

(b) How much work an operation performs

(c) Both the answer and its correctness guarantee

Reducing the probability of error Suppose a decision algorithm errs with probability at most 1/3. Run it an odd number k of times independently and take a majority vote. Its error probability becomes $e^{-\Omega(k)}$.

If an equality test only makes false-positive errors, each with probability at most δ, we can instead accept only when all k independent runs accept. The new error bound is δ^k.

Why is it important to use fresh randomness in the repeated runs?

2 Collisions

A *collision* is a pair of distinct keys x, y with $h(x) = h(y)$. If $|U| > B$, can we choose a function with no collisions at all?

Counting colliding pairs Fix distinct keys $x_1, \ldots, x_n$. For now, imagine that each key independently chooses a uniformly random bucket. Let I_{ij} be 1 if x_i and x_j collide, and 0 otherwise. Then

$$C = \sum_{i<j} I_{ij}$$

counts the number of colliding pairs.

For a fixed pair, $\mathbb{E}[I_{ij}] = 1/B$. Therefore,

$$\mathbb{E}[C] = \sum_{i<j} \mathbb{E}[I_{ij}] = \binom{n}{2} \frac{1}{B}.$$

The birthday effect Under this independent uniform model, for $n \leq B$,

$$\Pr[\text{no collision}] = \prod_{i=0}^{n-1} \left(1 - \frac{i}{B}\right) \approx e^{-n(n-1)/(2B)}.$$

This approximation is accurate when $n = O(\sqrt{B})$ and B is large. Thus collisions become likely around $n = \sqrt{B}$, even though most buckets are still empty.

Question 3. If $n = 1000$ and $B = 10^6$, approximately how many colliding pairs do we expect? Does a collision somewhere make every search slow?

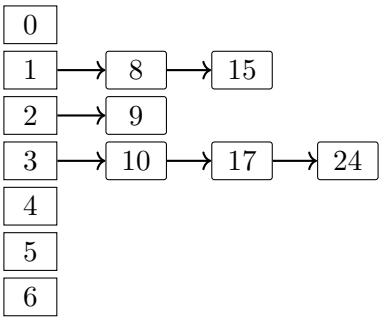
Resolving collisions by chaining

Each bucket stores a list of the keys that hash to it. To search for x, scan bucket $h(x)$ and compare x against the stored keys.

Example Let $B = 7$, let $h(x) = x \bmod 7$, and insert

$$10,\ 17,\ 24,\ 8,\ 15,\ 9.$$

The table below displays each chain in insertion order.



Question 4. How many stored keys does each search compare?

(a) SEARCH(24): 3

(b) SEARCH(31): 3

(c) SEARCH(9): 1

Deleting a key How would the table change after DELETE(17)?

The *load factor* is $\alpha = n/B$, the average number of keys per bucket. Chaining allows $\alpha > 1$, but longer chains mean more work.

3 Universal Hashing

The function $h(x) = x \bmod B$ sends every multiple of B to bucket zero. Could a different deterministic function avoid all bad sets of keys?

Theorem 1. *If $|U| > (n-1)B$, every function from U to B buckets has n distinct keys that all hash to the same bucket.*

Finding a bad set Some bucket has at least

$$\left\lceil \frac{|U|}{B} \right\rceil \geq n$$

preimages. Which principle gives this conclusion?

Choose n of these keys. A search through their chain can require $\Theta(n)$ comparisons.

Choosing a function at random A family $\mathcal{H}$ is *universal* if, for every distinct $x, y \in U$, a uniformly random $h \in \mathcal{H}$ satisfies

$$\Pr_h[h(x) = h(y)] \leq \frac{1}{B}.$$

We choose one function and use it throughout the table.

Question 5. Which statement does universality guarantee?

(a) Every bucket gets exactly n/B keys

(b) There are no collisions with high probability

(c) Each fixed pair collides with probability at most $1/B$

To be clear. The keys and queries in our analysis are fixed independently of the sampled function. Choosing keys after seeing the function need not satisfy the same guarantee.

Bounding the expected search time

Lemma 2. *With universal hashing and separate chaining, a fixed search has expected running time $O(1+\alpha)$, where $\alpha = n/B$.*

First, consider an unsuccessful search Let S be the fixed set of n stored keys and let $x \notin S$ be the query. The number of stored keys in its bucket is

$$C_x = \sum_{y \in S} \mathbf{1}[h(y) = h(x)].$$

By linearity of expectation,

$$\mathbb{E}[C_x] = \sum_{y \in S} \Pr[h(y) = h(x)] \leq \sum_{y \in S} \frac{1}{B} = \frac{n}{B}.$$

Hashing the key costs $O(1)$, and scanning the chain costs $O(C_x)$. Thus, the expected total is $O(1 + n/B)$.

Next, consider a successful search If $x \in S$, its chain contains x itself plus the other keys colliding with x. The expected chain length is at most

$$1 + \frac{n-1}{B}.$$

This gives the same asymptotic upper bound.

Question 6. Which step required independence between different collision events?

(a) Linearity of expectation

(b) Applying the collision bound to each pair

(c) Neither step

Return to the birthday effect With one million keys and one million buckets, there may be many collisions in the whole table. Why can a fixed search still have expected $O(1)$ cost?

4 Open Addressing

Chaining uses a separate list for each bucket. Can we instead store every key directly in the array?

If the first location is occupied, inspect additional locations in a *probe sequence*. A search must follow the same sequence as insertion.

Linear probing For $i = 0, 1, \ldots, B-1$, inspect

$$h(x, i) = (h(x) + i) \bmod B.$$

Before we introduce deletion, a search stops when it finds the key, reaches an empty slot, or has made B probes. An insertion searches along the same sequence and uses an empty slot if the key is absent.

Fill in the table Let $B = 11$, let $h(x) = x \bmod 11$, and insert

$$10,\ 21,\ 32,\ 4,\ 15.$$

Index	0	1	2	3	4	5	6	7	8	9	10
Key	21	32			4	15					10

Question 7. How many slots does SEARCH(43) inspect?

(a) One

(b) Three

(c) Four

(d) Eleven

Why might runs of occupied slots grow? A key hashing anywhere inside a run must pass through part of it. Inserting that key can make the run longer. This is called primary clustering.

A full table cannot accept a new key until it is enlarged. Stop after B probes rather than looping forever. Low load alone does not prove fast linear probing for an arbitrary hash function.

What happens when we delete a key?

Return to the previous table. Delete 21 from slot 0 and mark the slot empty. Now trace SEARCH(32).

We need to distinguish three states:

- Occupied: compare the stored key with the query.
- Never used: an unsuccessful search may stop.
- Deleted: an unsuccessful search must continue.

A deleted marker is called a *tombstone*.

Inserting after deletions Remember the first tombstone, but keep searching until finding the key, a never-used slot, or completing B probes. If the key is found, update its value. Otherwise, insert into the first remembered tombstone if one exists; if not, use the never-used slot. If neither exists, resize the table.

Why not insert immediately when we encounter a tombstone?

A different probe sequence Double hashing uses

$$h(x, i) = \big(h_1(x) + i\, h_2(x)\big) \bmod B.$$

With $B = 10$, start at slot 0 and use step size 2. Which slots are visited?

To visit all slots, the step must be relatively prime to B. When B is prime, any step in $\{1, \ldots, B-1\}$ works.

Many tombstones can make searches expensive even when few live keys remain. Rebuilding the table removes them.

5 Resizing and Amortized Analysis

As we insert more keys, the load factor $\alpha = n/B$ increases. We can keep it bounded by doubling the table size when it exceeds a fixed threshold.

What must be moved? Can we copy the old array into the first half of the new array and leave all keys where they were?

For chaining, we can rebuild by placing each key directly into its new list. Moving n keys costs $O(n)$, so a single insertion can be expensive.

Count the work over many insertions Start with a constant-size table and perform N insertions. Capacities grow geometrically. Up to constant factors, the total number of keys moved is bounded by

$$1 + 2 + 4 + \cdots + 2^{\lfloor \log_2 N \rfloor} < 2N.$$

The total rebuilding work is therefore $O(N)$, or $O(1)$ amortized per insertion.

Question 8. Which kind of averaging is used in each statement?

(a) Average over the random hash function: expected time

(b) Spread rebuilding costs across many operations: amortized time

With universal hashing, bounded load, and a fixed operation sequence independent of the randomness, chaining has expected amortized $O(1)$ cost per operation under a suitable resizing policy.

Avoiding repeated rebuilds What could happen if we double above load 1/2 and halve below load 1/2?

6 Fingerprints and String Equality

Suppose two computers hold strings x and y of length m. How can they check whether the strings are equal without sending all m characters?

They can agree on a randomly chosen hash function and exchange short fingerprints.

What can equal hashes tell us? For every function h,

$$x = y \implies h(x) = h(y).$$

Thus, different hashes certify that the strings are different. The converse can fail because of collisions.

Question 9. We output "equal" exactly when the fingerprints agree. Which error is possible?

(a) Rejecting two equal strings

(b) Accepting two unequal strings

(c) Both kinds of error

What work have we saved? We still have to read a string to compute its fingerprint. The benefit can be less communication, a compact summary, or faster repeated comparisons.

A related problem: string matching Given a text T of length n and a pattern P of length $m \leq n$, report all positions where P occurs. There are $n - m + 1$ candidate windows. Comparing every window with P takes $O(nm)$ time in the worst case.

We will compare the pattern's fingerprint with each window's fingerprint. The next question is how to compute all those fingerprints efficiently.

Turning a string into a polynomial

Choose a prime q and encode each alphabet symbol by a distinct integer in $\{0, \ldots, q-1\}$. For a base $b \in \{0, \ldots, q-1\}$, define

$$h_b(S) = \left(\sum_{j=0}^{m-1} s_j b^{m-1-j} \right) \bmod q.$$

We compare strings of the same length. For variable-length strings, compare the lengths too.

Example: digits For $S = 314$, base 10, and modulus 101,

$$h_{10}(314) = (3 \cdot 10^2 + 1 \cdot 10 + 4) \bmod 101 = 11.$$

Example: characters Map $a = 1, b = 2, \ldots, z = 26$. Using numeric base 31,

$$h_{31}(\texttt{cab}) = (3 \cdot 31^2 + 1 \cdot 31 + 2) \bmod 101 = 88.$$

Computing one fingerprint Start with $H = 0$. Read the characters from left to right. After reading a new character s, update

$$H \leftarrow (bH + s) \bmod q.$$

This is Horner's rule. It uses $O(m)$ modular arithmetic operations.

Question 10. Does choosing the fixed numbers $b = 31$ and $q = 101$ make this a randomized hash function?

(a) Yes, because modular arithmetic is unpredictable

(b) No, we have not made a random choice

(c) Yes, because different strings can collide

Updating a sliding window

Number text positions from 0. Let H_i be the hash of the length-m window starting at position i:

$$H_i = \sum_{j=0}^{m-1} t_{i+j} b^{m-1-j} \pmod{q}.$$

Precompute $a = b^{m-1} \bmod q$.

Remove the old character The character leaving the window is t_i. Subtract its contribution:

$$H_i - t_i a.$$

Shift the remaining characters and add the new one Multiply by the base, then add the new rightmost character t_{i+m}:

$$H_{i+1} = \big(b(H_i - t_i a) + t_{i+m}\big) \bmod q.$$

Each update uses only a constant number of modular operations.

Try one update Use text 3141, window length 3, base 10, and modulus 101. We know that $H_0 = 11$ and $a = 100$. Compute

$$H_1 = (10(11 - 3 \cdot 100) + 1) \bmod 101 = 40.$$

How can you check this answer directly?

Question 11. How much time do we need for each task?

(a) The first window and the power a: $O(m)$

(b) Each later window: $O(1)$

(c) All windows together: $O(n + m)$

These bounds assume constant-time arithmetic modulo q. After a subtraction, reduce to a value in $\{0, \ldots, q-1\}$; a language's remainder operation may return a negative value.

7 The Karp–Rabin Algorithm

Fix a prime q larger than all symbol encodings. Choose b uniformly from $\{0, \ldots, q-1\}$ and use the same base for the pattern and all text windows.

KARP–RABIN-CANDIDATES(T, P)

Let $n = |T|$ and $m = |P|$, where $1 \le m \le n$
Compute $H_P = h_b(P)$ and $H = h_b(t_0 \cdots t_{m-1})$
Compute $a = b^{m-1} \bmod q$
for $i = 0, 1, \ldots, n - m$ **do**
  if $H = H_P$ **then**
    Report i as a candidate occurrence
  end if
  if $i < n - m$ **then**
    $H \leftarrow \big(b(H - t_i a) + t_{i+m}\big) \bmod q$
  end if
end for

First, account for the running time Computing the initial hashes and power takes $O(m)$ time. There are at most n iterations, each taking $O(1)$ time. The total is therefore $O(n + m)$, including reporting the candidates.

Theorem 3. *Every true occurrence is reported. The algorithm can only err by reporting an extra occurrence.*

Why can a true occurrence never be missed? At a true occurrence, the pattern and window are identical strings. Therefore, their fingerprints agree for every choice of b.

Question 12. How could we make every reported occurrence exact?

(a) Check the actual characters at each candidate position

(b) Use any fixed prime modulus

(c) Stop after the first matching fingerprint

Why are collisions unlikely?

Let x and y be fixed, distinct strings of length m. Over the field of integers modulo q, form the difference polynomial

$$D(z) = \sum_{j=0}^{m-1} (x_j - y_j) z^{m-1-j}.$$

What do we know about this polynomial? Since $x \neq y$ and symbol encodings are distinct modulo q, at least one coefficient is nonzero. The degree is at most $m - 1$.

Lemma 4 (Polynomial root bound)**.** *A nonzero polynomial of degree at most r over a field has at most r roots.*

When do the hashes collide? The equality $h_b(x) = h_b(y)$ holds exactly when $D(b) = 0$. There are q equally likely bases, and at most $m - 1$ bad ones. Thus,

$$\Pr_b[h_b(x) = h_b(y)] \leq \min\{1, (m-1)/q\}.$$

What about every window in the text? There are at most $n - m + 1$ unequal windows. A union bound gives

$$\Pr[\text{at least one false occurrence}] \leq \frac{(n-m+1)(m-1)}{q}.$$

We can make this at most δ by choosing a prime $q \geq (n - m + 1)(m - 1)/\delta$, also large enough for the alphabet.

Question 13. Do the different window-collision events have to be independent for this union bound to hold?

(a) Yes

(b) No

A large fixed modulus alone does not give a collision probability for arbitrary inputs. Here the probability comes from the randomly chosen base.

What does verification cost?

Suppose we compare all m characters whenever a window's fingerprint matches the pattern's fingerprint. Let c be the number of candidates.

Account for the extra work The total running time is bounded by

$$O(n + m + mc).$$

The answer is always correct. Randomness now affects how much verification work is spent on false candidates.

Let r be the number of true occurrences and F the number of false candidates. Then $c = r + F$, and linearity of expectation gives

$$\mathbb{E}[F] \leq \frac{(n - m + 1)(m - 1)}{q}.$$

Consequently,

$$\mathbb{E}[\text{time}] = O\left(n + m + mr + \frac{m(n - m + 1)(m - 1)}{q}\right).$$

Can true matches be expensive? Let T consist of n copies of `a`, and let P consist of m copies of `a`. How many true occurrences are there?

$$r = n - m + 1.$$

How much time does checking every occurrence from scratch take?

Question 14. Does this simple exact variant guarantee expected $O(n + m)$ time?

(a) Yes, because collisions are unlikely

(b) No, because even true matches can require repeated work

Algorithms such as Knuth–Morris–Pratt find all occurrences in $O(n + m)$ time without this repeated verification. Hashing is useful for compact summaries, flexible comparisons, and filtering candidates.

8 What Does Hashing Guarantee?

Question 15. A system stores ten million document identifiers and repeatedly compares selected document contents. How could hashing help with each task? Where do collisions affect time, and where could they affect correctness?

Return to the guarantees For each claim, state the missing condition or explain why it is false.

- "A universal hash table has constant-time operations."
- "A large modulus makes collisions impossible."
- "Deleting a key in linear probing means marking its slot empty."
- "Verifying every Karp–Rabin candidate preserves linear time."

Hashing compresses a key or a string. Randomness lets us analyze collisions. The surrounding algorithm determines whether those collisions cost extra work or introduce a probability of error.

Optional Extension: Constructing a Universal Family

How can we choose a universal hash function without storing an independent random value for every possible key?

Let the number of buckets be a prime p. Represent keys as distinct vectors $x = (x_1, \ldots, x_\ell)$ over $\mathbb{F}_p$, the field of integers modulo p. Choose $a_1, \ldots, a_\ell$ independently and uniformly from $\mathbb{F}_p$, and set

$$h_a(x) = \sum_{j=1}^{\ell} a_j x_j \pmod{p}.$$

Theorem 5. *For distinct keys x, y, $\Pr_a[h_a(x) = h_a(y)] = 1/p$.*

Find a coordinate where the keys differ Since $x \neq y$, there is an index k with $x_k \neq y_k$. Fix all coefficients except a_k. A collision requires

$$a_k(x_k - y_k) = -\sum_{j \neq k} a_j(x_j - y_j) \pmod{p}.$$

Why does this equation determine exactly one value of a_k?

There are p equally likely choices of a_k, and exactly one causes a collision. The collision probability is therefore $1/p$.

Try the construction Take $p = 7$, $a = (2, 3)$, $x = (1, 4)$, and $y = (3, 1)$. Compute

$$h_a(x) = 0, \qquad h_a(y) = 2.$$

Does this calculation alone prove universality?

The function stores ℓ coefficients and uses ℓ field operations. This is constant time when $\ell = O(1)$ and field arithmetic costs $O(1)$.

Optional Extension: Bloom Filters

Suppose we only need to test membership, and a small false-positive probability is acceptable. Can we use much less space than storing all the keys?

A *Bloom filter* stores an array of B bits, initially zero, and uses k hash functions.

- Insert x: set bits $h_1(x), \ldots, h_k(x)$ to 1.
- Query x: answer "possibly present" if all those bits are 1; otherwise answer "definitely absent."

Question 16. Assuming bits are never cleared, which error is possible?

(a) A false negative

(b) A false positive

(c) Both

How full does the bit array become? Under ideal independent uniform hashing, after inserting n distinct keys, a particular bit is still zero with probability

$$\left(1 - \frac{1}{B}\right)^{kn} \approx e^{-kn/B}.$$

Treating queried bit states as approximately independent gives the useful false-positive approximation

$$\left(1 - e^{-kn/B}\right)^k.$$

This last expression is not an exact formula for a finite bit array.

Why is deletion difficult? Can we delete x simply by clearing its k bits?

A database can check a small Bloom filter before reading a large file from disk. A negative answer avoids the read; a positive answer triggers an exact lookup. Here, false positives increase work without changing the final answer.